Linformer: Low-Rank Projection.

Self-attention costs $O(T^2)$ because it forms a full $T \times T$ score matrix. Linformer starts from a striking empirical fact — that score matrix is approximately *low rank* — and exploits it directly: project the length-T keys and values down to a fixed $k \ll T$ rows *before* attention. The score matrix shrinks to $T \times k$ and the whole operation becomes *linear* in T. Every number on the following pages is computed in full, nothing summarised or deferred.

THE EQUATION

$$\text{Linformer}(Q,K,V) = \text{softmax}\left(\frac{Q\,(EK)^\top}{\sqrt{d}}\right)(FV), \qquad E,F \in \mathbb{R}^{k\times T},\ \ k \ll T$$

Worked example. T=8, d=4, k=2: project K, V (8×4) to K', V' (2×4), form the 8×2 score matrix (vs. 8×8 full), softmax its rows, and read off the 8×4 output — 16 score entries instead of 64.

Read top to bottom: each step feeds the next.

Contents

1 The claim: attention is approximately low rank

Full scaled dot-product attention on a sequence of length T with head dimension d is

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \qquad Q,K,V \in \mathbb{R}^{T\times d}.$$

The score matrix $P = \text{softmax}(QK^\top/\sqrt{d}) \in \mathbb{R}^{T\times T}$ has T^2 entries; both forming it and multiplying it by V cost $O(T^2 d)$. Linformer's premise is empirical: across heads and layers the score matrix P is well approximated by a matrix of *rank* $r \ll T$. Its singular values decay fast, so almost all of P's "information" lives in a few directions. If P is effectively rank k, we are paying for T^2 numbers to represent something that needs only $\sim kT$.

A length-T sentence does not contain T *independent* things to attend to. Many tokens are redundant or carry overlapping context, so the $T \times T$ table of "who looks at whom" has lots of near-duplicate rows and columns — it is squashable. Linformer squashes the *keys and values* ahead of time: instead of comparing every query against all T keys, it compares each query against k learned *summaries* of the keys. With k fixed, the cost no longer grows with how long the sequence is.

2 The construction: project before attending

Introduce two learned projection matrices $E, F \in \mathbb{R}^{k\times T}$. Apply them along the *sequence* axis to compress T rows into k:

$$K' = EK \in \mathbb{R}^{k\times d}, \qquad V' = FV \in \mathbb{R}^{k\times d}, \qquad \boxed{\text{Linformer}(Q,K,V) = \text{softmax}\left(\frac{Q\,K'^{\top}}{\sqrt{d}}\right)V'} \tag{1}$$

The queries $Q \in \mathbb{R}^{T\times d}$ are untouched, so there is still one output row per input token. What changes is the inner dimension of the comparison: the score matrix is now $Q\,K'^{\top} \in \mathbb{R}^{T\times k}$, not $T \times T$.

Symbol	Shape	Role	Cost to form
Q	$T \times d$	queries (one per token)	—
K, V	$T \times d$	full keys / values	—
E, F	$k \times T$	learned length-projections	—
$K' = EK$	$k \times d$	compressed keys	$O(kTd)$
$V' = FV$	$k \times d$	compressed values	$O(kTd)$
$QK'^{\top}$	$T \times k$	score matrix (was $T \times T$)	$O(Tkd)$
output	$T \times d$	per-token result	$O(Tkd)$

3 Deriving the complexity drop

Count the dominant matrix multiplications.

3.1 Full attention: $O(T^2 d)$

Forming $QK^{\top}$ multiplies $(T \times d)$ by $(d \times T)$: that is $T^2 d$ multiply–adds and produces T^2 scores. Multiplying $P\,V$ multiplies $(T \times T)$ by $(T \times d)$: another $T^2 d$. Total $\approx 2T^2 d = O(T^2 d)$ — *quadratic* in sequence length.

3.2 Linformer: $O(Tkd)$

Four steps, each linear in T:

$$\underbrace{EK}_{kTd} + \underbrace{FV}_{kTd} + \underbrace{QK'^{\top}}_{Tkd} + \underbrace{P'V'}_{Tkd} = 4\,Tkd = O(Tkd).$$

With k held *fixed* as T grows, this is $O(T)$ — *linear*. The speed-up ratio is

$$\boxed{\frac{O(T^2 d)}{O(Tkd)} = \frac{T}{k}} \tag{2}$$

so at $k = 256$ a $T = 32768$ sequence is $T/k = 128\times$ cheaper in the score path.

4 Worked computation (T=8, d=4, k=2)

Inputs (chosen to keep arithmetic exact):

$$Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 2 & 1 & 0 & 1 \\ 0 & 0 & 2 & 1 \end{bmatrix}.$$

Projections (each row is a mean over four tokens, so $E, F \in \mathbb{R}^{2\times 8}$):

$$E = \tfrac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}, \qquad F = \tfrac{1}{2}\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

4.1 Step 1 — compress keys and values ($k \times d$)

$K' = EK$ averages the first four key rows (row 1) and last four (row 2); $V' = FV$ averages the odd-indexed and even-indexed value rows:

$$K' = EK = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1.5 \end{bmatrix}, \qquad V' = FV = \begin{bmatrix} 3 & 1 & 1.5 & 0.5 \\ 0 & 1.5 & 1.5 & 2.5 \end{bmatrix} \quad (2 \times 4).$$

For instance row 1 of K' is $\frac{1}{4}(K_1{+}K_2{+}K_3{+}K_4) = \frac{1}{4}[2,2,2,2] = [1,1,1,1]$, and the last entry of K' row 2 is $\frac{1}{4}(0{+}1{+}1{+}1) + \frac{1}{4}(1){=}1.5$ after summing $K_5..K_8$.

4.2 Step 2 — score matrix $T \times k$ (not $T \times T$)

$S = QK'^{\top}/\sqrt{d}$ with $\sqrt{d} = 2$. Each query (a 4-vector) dots the two compressed keys:

$$QK'^{\top} = \begin{bmatrix} 2 & 2 \\ 2 & 2.5 \\ 2 & 2 \\ 2 & 2.5 \\ 2 & 2.5 \\ 2 & 2 \\ 3 & 3 \\ 2 & 2.5 \end{bmatrix}, \qquad S = \tfrac{1}{2}\,QK'^{\top} = \begin{bmatrix} 1 & 1 \\ 1 & 1.25 \\ 1 & 1 \\ 1 & 1.25 \\ 1 & 1.25 \\ 1 & 1 \\ 1.5 & 1.5 \\ 1 & 1.25 \end{bmatrix} \in \mathbb{R}^{8\times 2}.$$

This 8×2 block holds 16 numbers; the full 8×8 score matrix would hold 64. We never materialise the missing 48.

4.3 Step 3 — softmax over the k columns

Row-wise softmax of S (only two entries per row). For a row $[1, 1]$ the weights are exactly $[0.5, 0.5]$; for $[1, 1.25]$ they are $[\,e^0/(e^0+e^{0.25}),\ e^{0.25}/(\cdot)\,] = [0.4378, 0.5622]$:

$$P' = \mathrm{softmax}(S) = \begin{bmatrix} 0.5 & 0.5 \\ 0.4378 & 0.5622 \\ 0.5 & 0.5 \\ 0.4378 & 0.5622 \\ 0.4378 & 0.5622 \\ 0.5 & 0.5 \\ 0.5 & 0.5 \\ 0.4378 & 0.5622 \end{bmatrix} \in \mathbb{R}^{8\times 2}.$$

Each row sums to 1 over the $k = 2$ *summaries*, not over $T = 8$ tokens.

4.4 Step 4 — output $T \times d$

$O = P'V' \in \mathbb{R}^{8\times 4}$. Row 1 (weights $[0.5, 0.5]$) is the average of V''s two rows: $\frac{1}{2}([3, 1, 1.5, 0.5] + [0, 1.5, 1.5, 2.5]) = [1.5, 1.25, 1.5, 1.5]$. Row 2 (weights $[0.4378, 0.5622]$): $0.4378\,[3, 1, 1.5, 0.5] + 0.5622\,[0, 1.5, 1.5, 2.5] = [1.3135, 1.2811, 1.5, 1.6244]$.

$$O = \begin{bmatrix} 1.5 & 1.25 & 1.5 & 1.5 \\ 1.3135 & 1.2811 & 1.5 & 1.6244 \\ 1.5 & 1.25 & 1.5 & 1.5 \\ 1.3135 & 1.2811 & 1.5 & 1.6244 \\ 1.3135 & 1.2811 & 1.5 & 1.6244 \\ 1.5 & 1.25 & 1.5 & 1.5 \\ 1.5 & 1.25 & 1.5 & 1.5 \\ 1.3135 & 1.2811 & 1.5 & 1.6244 \end{bmatrix} \in \mathbb{R}^{8\times 4}.$$

Shapes tracked end to end: $K, V\,(8\times4) \to K', V'\,(2\times4) \to S\,(8\times2) \to P'\,(8\times2) \to O\,(8\times4)$.

The heatmap shades the 8×2 attention weights P' — the entire score matrix Linformer ever forms (intensity scaled by value, capped at `indigo!80`):

	summary 1	summary 2
row 1	0.500	0.500
row 2	0.438	**0.562**
row 7	0.500	0.500

The associativity-free shortcut is purely *dimensional*: by replacing the T keys/values with k learned summaries *before* the softmax, every matrix that touches the sequence axis has size $T \times k$ or $k \times d$ instead of $T \times T$. With k fixed, attention's cost grows like T, not T^2 — a T/k reduction in the score path.

5 Cost across sequence lengths (k=256)

Score-matrix entries (and, up to the constant $4d$, the multiply count) for full attention T^2 versus Linformer Tk:

T	**full** T^2	**Linformer** Tk	**ratio** T/k
512	262,144	131,072	2×
1024	1,048,576	262,144	4×
2048	4,194,304	524,288	8×
4096	16,777,216	1,048,576	16×
8192	67,108,864	2,097,152	32×
16384	268,435,456	4,194,304	64×
32768	1,073,741,824	8,388,608	128×

Full attention quadruples each time T doubles; Linformer merely doubles. The gap T/k widens without bound.

6 Properties / consequences that matter

1. **Linear in T (fixed k).** Time and memory are $O(Tkd)$; the score matrix never exceeds $T \times k$, so the $O(T^2)$ activation-memory wall of full attention disappears.
2. **Deterministic, not stochastic.** E, F are learned weight matrices — the same every forward pass. There is no sampling, so outputs are reproducible (contrast Performer in Topic 2).
3. **Parameter sharing.** The original Linformer ties E and F (and shares them across heads/layers) to keep the extra kT parameters small.
4. **Encoder-shaped.** The projections act on the *full* key/value set at once, so vanilla Linformer suits bidirectional encoders; causal decoding (a growing prefix) needs extra care because E, F assume a fixed T.

The fixed k is a hard information bottleneck. Compressing T keys into k summaries discards exactly the fine-grained, token-to-token detail that long-range *retrieval* tasks need ("find the one sentence that mentions X"). On such needle-in-a-haystack benchmarks Linformer typically loses **5–12 points** versus full attention, and the loss *worsens* as T grows with k held fixed. The cure is to let k scale with T (e.g. $k \propto \log T$ or a small fraction of T) — but that erodes the clean "$O(T)$ at constant k" story.

7 Where this sits in the track

Linformer is the *deterministic, low-rank* answer to attention's $O(T^2)$ cost: shrink the sequence axis with learned projections. Topic 2 (Performer) reaches the same $O(T)$ goal by a different route — a *kernel feature map* plus the associativity reordering $(QK^\top)V = Q(K^\top V)$ — which keeps full length flexibility and supports causal decoding, at the price of approximation variance. Together they bracket the design space of efficient attention covered in Phase 5.

A Reproducible (NumPy)

Inputs: the integer $Q, K, V \in \mathbb{R}^{8\times 4}$ and averaging projections $E, F \in \mathbb{R}^{2\times 8}$ above. The snippet reproduces K', V', the 8×2 scores, the softmax weights, the output, and the entry-count comparison.

```
import numpy as np
Q=np.array([[1,0,1,0],[0,1,0,1],[1,1,0,0],[0,0,1,1],
            [1,0,0,1],[0,1,1,0],[1,1,1,0],[0,1,0,1]],float)
K=np.array([[1,1,0,0],[0,1,1,0],[0,0,1,1],[1,0,0,1],
            [1,0,1,0],[0,1,0,1],[1,1,1,1],[0,0,0,1]],float)
V=np.array([[2,0,1,0],[0,2,0,1],[1,0,2,0],[0,1,0,2],
            [1,1,0,0],[0,0,1,1],[2,1,0,1],[0,0,2,1]],float)
E=np.array([[1,1,1,1,0,0,0,0],[0,0,0,0,1,1,1,1]],float)/2
F=np.array([[1,0,1,0,1,0,1,0],[0,1,0,1,0,1,0,1]],float)/2
T,d,k=8,4,2
Kp,Vp = E@K, F@V                       # (2,4) each
S = Q@Kp.T/np.sqrt(d)                  # (8,2)
P = np.exp(S-S.max(1,keepdims=True)); P/=P.sum(1,keepdims=True)
O = P@Vp                               # (8,4)
print(Kp); print(Vp); print(np.round(P,4)); print(np.round(O,4))
print("score entries: full", T*T, "linformer", T*k)   # 64 16
# Kp row1 = [1,1,1,1]; P row2 = [0.4378 0.5622]
# O row2  = [1.3135 1.2811 1.5 1.6244]
```